

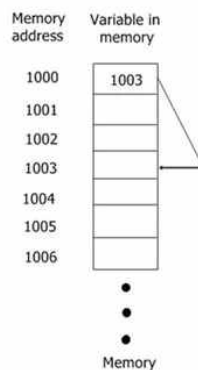
C언어 5일차 과제1 - 포인터 보고서

로봇학부, 2026406041, 김도훈

1. 포인터란 무엇인가?

1-1 포인터의 기본 개념

포인터는 메모리의 주소를 저장하는 변수이다. 여기서 메모리란 수많은 칸으로 나뉜 거대한 바이트 배열로 볼 수 있다. 우리가 프로그래밍을 하면서 작성하는 변수가 이 메모리에 적절한 크기의 공간을 할당 받아 저장된다. 메모리의 주소는 공간의 주소인 것이다.



위의 사진에서 보면 1003이라는 값이 메모리에 적절한 공간의 크기를 할당 받아 저장되어 있다. 1003이 저장되어있는 변수의 주소는 1000이 된다. 처음 배우는 사람의 입장에서는 “왜 굳이 변수의 주소를 또 다른 변수에 저장하는지 의문이 들것이다.” 나도 그랬다. 우리가 포인터를 배우는 이유는 메모리 직접 제어하여 효율적이게 데이터를 전달하고 메모리를 관리하기 위해서이다. 말이 좀 어렵다. 본인 학부 1학년이 이해한 C언어에서 포인터를 배우는 이유는 Call by Reference와 malloc이다. 첫 번째 Call by Reference는 포인터를 사용하여 변수의 주소에 있는 값을 변경한다. 즉 원본을 변경할 수 있다는 것이다. 두 번째 malloc은 프로그램이 실행되는 도중에 필요한 만큼 메모리를 할당 받고 해제할 때 포인터가 필수적이다. 이렇게 학부 1학년이 이해한 C언어에서 포인터를 사용하는 두 가지 이유라고 본다.

1-2 포인터의 사용

```
int n = 100; // 변수의 선언
int* ptr = &n; //포인터의 선언
```

위의 코드는 자료형이 int인 변수 n에 100을 저장했다. 위에서 설명한 메모리의 관점으로 설명하며 메모리의 4바이트(int) 공간을 할당 받아 그 공간에 정수 100을 저장한 것이다. 그리고 주소 연산자를 사용해(&) n의 주소를 가져오고 이 주소를 포인터 연산자(*)를 붙인 ptr 포인터 변수에 저장한 것이다.

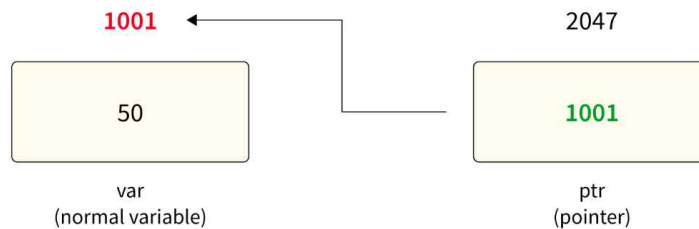
추가적으로 임의의 변수의 주소를 저장할 포인터 변수를 선언할 때 *을 붙이는데 이 별은 변수에 앞에 붙이면 된다.

```
int *ptr;
int* ptr;
int * ptr;
```

모든 똑같은 뜻이다. 이제 이 포인터 활용해보자.

```
int n = 100;
int* ptr = &n;
*ptr = 20;
```

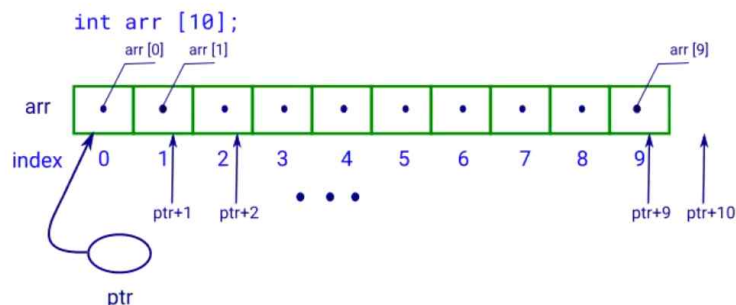
위의 코드를 이어서 * 역참조 연산자를 사용해 n의 값을 20으로 변경한 코드이다. 이 코드를 처음 봤을 때 뭘 소리인가 싶었다. 여기서 날 이해한게 해준 포인터는 포인터 변수를 선언할 때 사용한 * 과 n의 값을 변경할 때 사용한 *은 다른 의미라는 것이다. 포인터 변수를 선언할 때 사용하는 *은 포인터 변수를 선언한다. 라는 표시 같은 것이고 값을 변경할 때 사용하는 *은 역참조, Dereference 연산자이다. 이 연산자는 포인터 변수가 가리키는 메모리 주소에 직접 접근하여 그 안의 값을 읽는 기능을 한다. 이 연산자를 사용함으로써 n의 주소를 저장하여 이 주소의 원본값을 알 수 있고 수정할 수 있는 것이다.



역참조 연산자가 그림에서의 화살표 역할을 한다고 생각하면 이해하기 쉽다.

2. 포인터와 배열

2-1 배열



```
int array[3] = {1,2,3};
```

포인터와 배열은 아주 유사한 성격을 가지고 있다. 일단 위에 int형 값을 3개 가지는 배열을 선언한 코드가 있다. 사실 배열의 이름은 배열의 시작 주소를 의미하기도 한다. `arrar = &array[0]` 이다. 이렇게 배열의 이름은 그 배열의 인덱스 0의 주소를 의미한다. 포인터 방식으로 접근하면 배열 이름은 배열의 시작 주소이고 *(역참조 연산자)를 응용하면 주소의 본래의 값을 할 수 있다. 그러므로 `*array = 1`이다. 또한 배열 속 다른 값들에 접근하기 위해서는 `array[1] = 2`, `array[2] = 3` 이렇게 인덱스를 활용했다. 이것을 포인터 방식으로 접근하면 `*array = *(array + 0)`과 같은 의미이다. 그래서 `*(array + 1) = 2`, `*(array + 2) = 3`이 된다. 즉 `arr[i] = *(array + i)`이다.

2-2 문자열

```
char str[6] = "Hello";           str[0] str[1] str[2] str[3] str[4] str[5]
                                   'H'   'e'   'l'   'l'   'o'   '\0'
```

C언어에서 문자열은 문자들을 연속해서 저장한 char 형 배열이다. 위 코드는 'H' , 'e' , 'l' , 'l' , 'o' 라는 다섯 개의 문자와 문자열의 끝을 나타내는 널 문자 '\0' 을 저장한다. 따라서 "Hello"는 다섯 글자이지만 널 문자까지 포함해야 하므로 배열의 크기는 6이 된다. 문자열도 배열이므로 배열의 이름인 str은 첫 번째 문자인 str[0]의 주소를 의미한다. 즉, 대부분의 식에서 str은 &str[0]과 같은 주소를 나타낸다. 따라서 str을 한 번 역참조한 *str은 첫 번째 문자인 'H'를 의미한다. (*str == 'H') 문자열의 다른 문자에 접근할 때도 배열의 인덱스 방식과 포인터 방식을 모두 사용할 수 있다. 예를 들어 str[1]은 두 번째 문자인 e를 의미하며, 포인터 방식으로는 *(str + 1)과 같이 표현할 수 있다.

```
str[0] == *(str + 0)   str[1] == *(str + 1)   str[2] == *(str + 2)   str[3] == *(str + 3)
str[4] == *(str + 4)   str[5] == *(str + 5)
```

즉 문자열에서도 다음 두 표현은 같은 의미를 가진다. `str[i] == *(str + i)`

문자열의 시작 주소는 문자 포인터에도 저장할 수 있다. `char *ptr = str;` 이때 ptr은 문자열의 첫 번째 문자인 str[0]을 가리킨다. 따라서 `ptr[0], *ptr, str[0]`은 모두 H를 의미한다. 또한 `ptr + 1`은 다음 문자의 주소를 나타내기 때문에 `*(ptr + 1)`을 사용하면 e에 접근할 수 있다.

```
*ptr == 'H'
*(ptr + 1) == 'e'
```

결국 C언어의 문자열은 특별한 자료형이 아니라, 마지막에 널 문자 '\0'이 들어 있는 문자 배열이다. 문자열이 배열로 구성되어 있기 때문에 배열의 인덱스 방식뿐만 아니라

포인터 연산을 이용해서도 각각의 문자에 접근할 수 있다.

3. 구조체 포인터

구조체에도 포인터 사용이 가능하다.

```
struct student{          struct student s1; // 구조체 호출
    char name[10];        struct student* ptr; // 구조체를 저장할 수 있는 포인터
    int score;            ptr = &s1; // 구조체의 주소를 포인터 변수에 저장
}; // 구조체 선언
```

일반적인 구조체는 이렇게 선언한다. 포인터를 사용하기 위해서 구조체를 저장할 수 있는 포인터 변수를 선언해주고 그 변수에 구조체 s1의 주소를 저장한다. 배열에서는 배열의 이름 자체가 시작 주소였다. 그래서 구조체 또한 비슷할 거 생각했지만 구조체는 &를 꼭 사용해야한다. 이것에 주의하자.

```
ptr —————> s1
      |
      |— name
      |— score
```

관계는 이렇다. 이 내용을 처음 접할 때 굳이 포인터를 사용해 구조체들 간의 관계를 복잡하게 만들 필요가 있나 싶었다. 하지만 함수에서 원본 구조체를 수정할 때 매우 유용하다. 다 이유가 있었다.

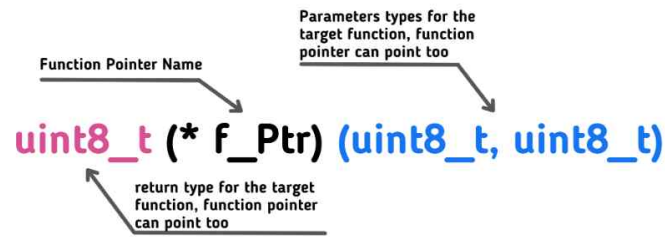
이렇게 ptr 변수에 구조체 주소를 저장했다면 포인터를 사용해 구조체의 멤버에 접근하여 값들을 수정할 수 있다.

```
s1.name = "dohun ";          (*ptr).name = "kimdohun ";
s1.score = 100;              (*ptr).score = 90;
(' .' 대신 -> 사용 가능)
```

구조체의 멤버에 접근할 때 ‘.’ 을 사용해 접근한다. 포인터 구조체도 똑같다. 포인터 변수에 역참조 연산자를 사용하여 멤버에 접근한다. 여기서도 ‘*’ 이 역참조 연산자라는 것을 잘 생각하면 s1.name과 (*ptr).name은 같은 의미이다. 포인터로 표현만 바꾼 것이다. 만약 (*ptr)을 사용하기 어렵다면 ->을 사용해 ptr -> name = "dohun" ; 으로 사용가능하다. 난 포인터를 배울 때 ->를 사용하는 것이 더 직관적이라 이 표현법을 주로 사용했다.

4. 함수 포인터

배열, 구조체, 함수 중 포인터를 어떻게 사용할지 이해가 잘 가지 않는 것이 함수였다. 하지만 함수도 그 함수가 실행되면 고유한 메모리 주소를 갖게 된다. 함수 포인터는 이 메모리 주소를 저장한다.



return_type (*pointer_name)(parameter_list);

- **return_type** → The return type of the function.
- **pointer_name** → Name of the function pointer.
- **parameter_list** → The function's parameter types.

함수 포인터를 선언하는 방법은 매우 까다롭다.

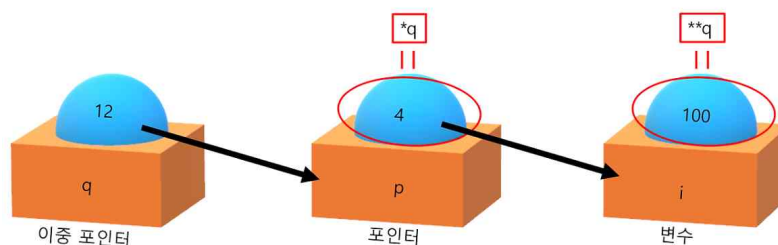
반환 자료형(*포인터 이름)(매개변수 타입) 으로 선언한다. 다음과 같다.

int add(int a, int b)	int (*ptr)(int, int);	int : 본 함수의 반환 값의 자료형
{		(*ptr) : 함수 포인터 이름
return a + b;		(int, int) : 매개변수 자료형
}		

이렇게 함수의 반환값 자료형과 매개변수의 자료형 및 개수를 똑같이 선언해야한다. 여기서 (*ptr) 이 괄호가 매우 중요하다. 이 괄호가 없다면 의도하는 함수 포인터가 아닌게 된다. 사용 시에는 int (*ptr)(int, int) = add; 로 함수 포인터 변수에 본 함수의 주소를 저장한다. 여기서도 배열과 똑같이 함수의 이름이 함수의 주소를 의미한다. (int (*ptr)(int, int) = &add; -> 사용 가능)

함수 포인터를 어디에 사용할지 의문이 생길 수 있다. 이 함수 포인터는 함수를 여러 함수 중 선택할 때, 함수를 다른 함수에 전달할 때 주로 사용한다.

5. 이중 포인터(포인터의 포인터)



이중 포인터는 말 그대로 포인터를 두 번 사용한 것이다. 어떠한 변수가 있을 때 그 변수의

주소를 가지고 있는 포인터 변수가 있고 이 포인터 변수의 주소를 가지고 있는 이중 포인터 변수가 있다.

```
int i = 100;
int* ptr = &i; // i의 주소
int** pptr = &ptr; // ptr의 주소
```

이중 포인터 선언 **두개를 사용해 선언한다. 포인터 변수에 &를 사용해 주소를 저장할 수 있다. 이중 포인터 또한 역참조 연산이 가능하다.

```
int i = 100;
int* ptr = &i;
int** pptr = &ptr;
```

출력은 다음과 같다.

```
printf("ptr: %p\n", ptr);
printf("*ptr: %d\n", *ptr);
printf("&ptr: %p\n", &ptr);
printf("pptr: %p\n", pptr);
printf("**pptr: %p\n", *pptr);
printf("&pptr: %p\n", &pptr);
printf("***pptr: %d\n", **pptr);
```

```
ptr: 0x7ffc6ed33fa4
*ptr: 100
&ptr: 0x7ffc6ed33fa8
pptr: 0x7ffc6ed33fa8
*pptr: 0x7ffc6ed33fa4
&pptr: 0x7ffc6ed33fb0
***pptr: 100
```

하나면 설명하면 **pptr은 역참조 연산을 두 번 실행한 것이다. 'pptr'은 'ptr'의 주소를 저장하고 있는 이중 포인터이다. '*pptr'처럼 한 번 역참조하면 'ptr'에 저장된 'i'의 주소가 나오고 '**pptr'처럼 한 번 더 역참조하면 'i'에 저장된 실제 값이 나온다.

pptr → ptr → i